Tallinn University of Technology

Forward Kinematics (FK)

Author: Jordan Thomas Brodie

Student Code: 243483

Individual variant: RPR8

Course: ATR0030 Robotics

October 5, 2026

1 Task

The purpose of this work was to create a forward kinematics model of my assigned manipulator, **variant RPR8**, in its home configuration where the joint variables are

$$A = 0, \qquad B = 0.$$

The program had to determine the positions and orientations of joints J1, J2, J3 and the end effector (EE), and then plot the complete manipulator together with its coordinate frames.

Before creating the MATLAB model, the coordinate frames were determined from a hand-made sketch. The z-axes of the joint frames were aligned with the corresponding revolute-joint axes or the direction of movement of the prismatic joint. The x- and y-axes were then selected according to the right-hand rule.

2 Initial Data

My assigned manipulator is the **RPR8** variant.

The link dimensions used in the MATLAB model are shown in Table 1.

Table 1: Initial link dimensions for variant RPR8

Parameter	Value	Description
L_0	3	Base offset
L_1	5	J1–J2 link length
$L_{2,\max}$	6	Maximum prismatic displacement
L_3	5	J3–EE link length

The home geometry was defined as follows:

- J0 to J1: translation of L_0 along $+z$;
- the J0 frame is rotated by $-\pi/2$ around its y-axis to obtain the orientation of J1;
- J1 to J2: translation of L_1 along local $+x$;
- J2 to J3: translation of $L_{2,\max}$ along local $-z$;
- J3 to EE: translation of L_3 along local $+y$.

3 Hand-Made Home Configuration Sketch

The Figure below shows my hand-made sketch used to define the joint positions and coordinate-frame orientations before creating the MATLAB model.

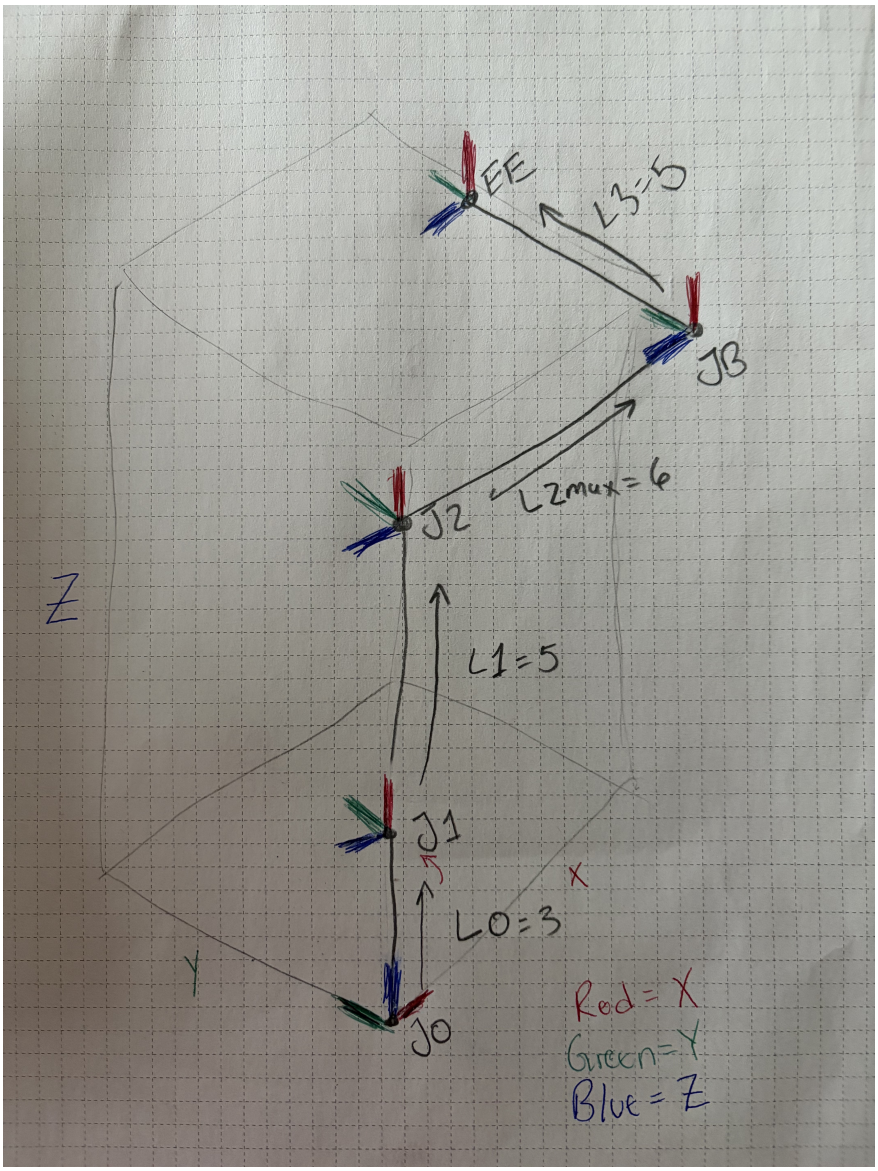


Figure 1: Hand-made sketch of the RPR8 home configuration.

4 Model Parameters

The geometry of the manipulator was represented using translation vectors and an axis-angle rotation vector.

The translation vectors are

$$T_{01} = \begin{bmatrix} 0 & 0 & L_0 \end{bmatrix},$$

$$T_{12} = \begin{bmatrix} L_1 & 0 & 0 \end{bmatrix},$$

$$T_{23} = \begin{bmatrix} 0 & 0 & -L_{2,\max} \end{bmatrix},$$

and

$$T_{3E} = \begin{bmatrix} 0 & L_3 & 0 \end{bmatrix}.$$

The J0 frame is rotated around its y-axis by $-\pi/2$:

$$R_{01} = \begin{bmatrix} 0 & 1 & 0 & -\pi/2 \end{bmatrix}.$$

MATLAB converts these translation and axis-angle vectors into homogeneous transformation matrices using the functions

`trvec2tform()` and `axang2tform()`.

The transformations are combined in sequence:

$${}^0T_1 = T_{01}R_{01},$$

$${}^0T_2 = {}^0T_1T_{12},$$

$${}^0T_3 = {}^0T_2T_{23},$$

and

$${}^0T_E = {}^0T_3T_{3E}.$$

These transformations describe the pose of every joint and the end effector relative to the base coordinate frame.

5 Calculated Joint Positions

For the home configuration, the resulting position vectors are

$$P_1 = \begin{bmatrix} 0 & 0 & 3 \end{bmatrix},$$

$$P_2 = \begin{bmatrix} 0 & 0 & 8 \end{bmatrix},$$

$$P_3 = \begin{bmatrix} 6 & 0 & 8 \end{bmatrix},$$

and

$$P_E = \begin{bmatrix} 6 & 5 & 8 \end{bmatrix}.$$

These positions correspond to the geometry shown in the MATLAB simulation.

Table 2: Joint and end-effector positions in the base frame

Point	x	y	z
J1	0	0	3
J2	0	0	8
J3	6	0	8
EE	6	5	8

6 MATLAB Program

The complete MATLAB program used to generate the RPR8 model is shown below.

```
clear; clc; close all

%% Jordan Brodie - RPR8 Forward Kinematics

% Link lengths - Variant 8
L0 = 3;
L1 = 5;
L2max = 6;
L3 = 5;

%% Translation and reorientation vectors

% J0 -> J1
T01 = [0 0 L0];

% Rotate J0 frame around y-axis to obtain J1 frame
R01 = [0 1 0 -pi/2];

% J1 -> J2: travel 5 units along local +x
T12 = [L1 0 0];

% J2 -> J3: travel 6 units along local -z
% L2 is the prismatic link for RPR8
T23 = [0 0 -L2max];

% J3 -> End Effector: travel 5 units along local +y
T3E = [0 L3 0];

%% Convert to homogeneous transformation matrices

T01H = trvec2tform(T01);
R01H = axang2tform(R01);
T12H = trvec2tform(T12);
T23H = trvec2tform(T23);
T3EH = trvec2tform(T3E);

%% RPR8 geometry model

J1 = T01H * R01H;
J2 = J1 * T12H;
J3 = J2 * T23H;
EE = J3 * T3EH;
```

```
%% Position vectors

P1 = tform2trvec(J1);
P2 = tform2trvec(J2);
P3 = tform2trvec(J3);
PE = tform2trvec(EE);

%% Display joint positions

fprintf('        P1         P2         P3         PE\n')
disp([P1(:) P2(:) P3(:) PE(:)])

%% Plot RPR8 manipulator

windowFunction;
title('Jordan Brodie RPR8 Forward Kinematics')

linksFunction(P1,P2,P3,PE)
labelsFunction(P1,P2,P3,PE)
framesFunction(J1,J2,J3,EE)

%% ============================================================
% USER-DEFINED FUNCTIONS
% ============================================================

%% Figure window
function windowFunction

    plot3(0,0,0,'o')

    hold on
    grid on
    axis equal

    xlabel('x')
    ylabel('y')
    zlabel('z')

    view(3)

end

%% Plot links
function linksFunction(P1,P2,P3,PE)

    % J0 -> J1
    plot3([0 P1(1)], ...
          [0 P1(2)], ...
          [0 P1(3)], ...
          'b-.','LineWidth',1.5)

    % J1 -> J2
    plot3([P1(1) P2(1)], ...
          [P1(2) P2(2)], ...
          [P1(3) P2(3)], ...
          'b-.','LineWidth',1.5)

    % J2 -> J3
```

```
    plot3([P2(1) P3(1)], ...
          [P2(2) P3(2)], ...
          [P2(3) P3(3)], ...
          'r-.','LineWidth',1.5)

    % J3 -> End Effector
    plot3([P3(1) PE(1)], ...
          [P3(2) PE(2)], ...
          [P3(3) PE(3)], ...
          'g-.','LineWidth',1.5)

end

%% Plot joint labels
function labelsFunction(P1,P2,P3,PE)

    d = 0.2;

    text(d,0,0,'J0')

    text(P1(1)+d, ...
         P1(2), ...
         P1(3)-d, ...
         'J1')

    text(P2(1)+d, ...
         P2(2), ...
         P2(3)-d, ...
         'J2')

    text(P3(1)+d, ...
         P3(2), ...
         P3(3)-d, ...
         'J3')

    text(PE(1)+d, ...
         PE(2), ...
         PE(3)-d, ...
         'EE')

end

%% Plot coordinate frames
function framesFunction(J1,J2,J3,EE)

    plotTransforms( ...
        tform2trvec(J1), ...
        tform2quat(J1))

    plotTransforms( ...
        tform2trvec(J2), ...
        tform2quat(J2))

    plotTransforms( ...
        tform2trvec(J3), ...
        tform2quat(J3))

    plotTransforms( ...
```

```
        tform2trvec(EE), ...
        tform2quat(EE))

end
```

7 Simulation Results

Figure 2 shows the home configuration generated by the MATLAB program.

The plot contains the base point J0, joints J1–J3, the end effector, the manipulator links, and the local coordinate frames at the joints and end effector.

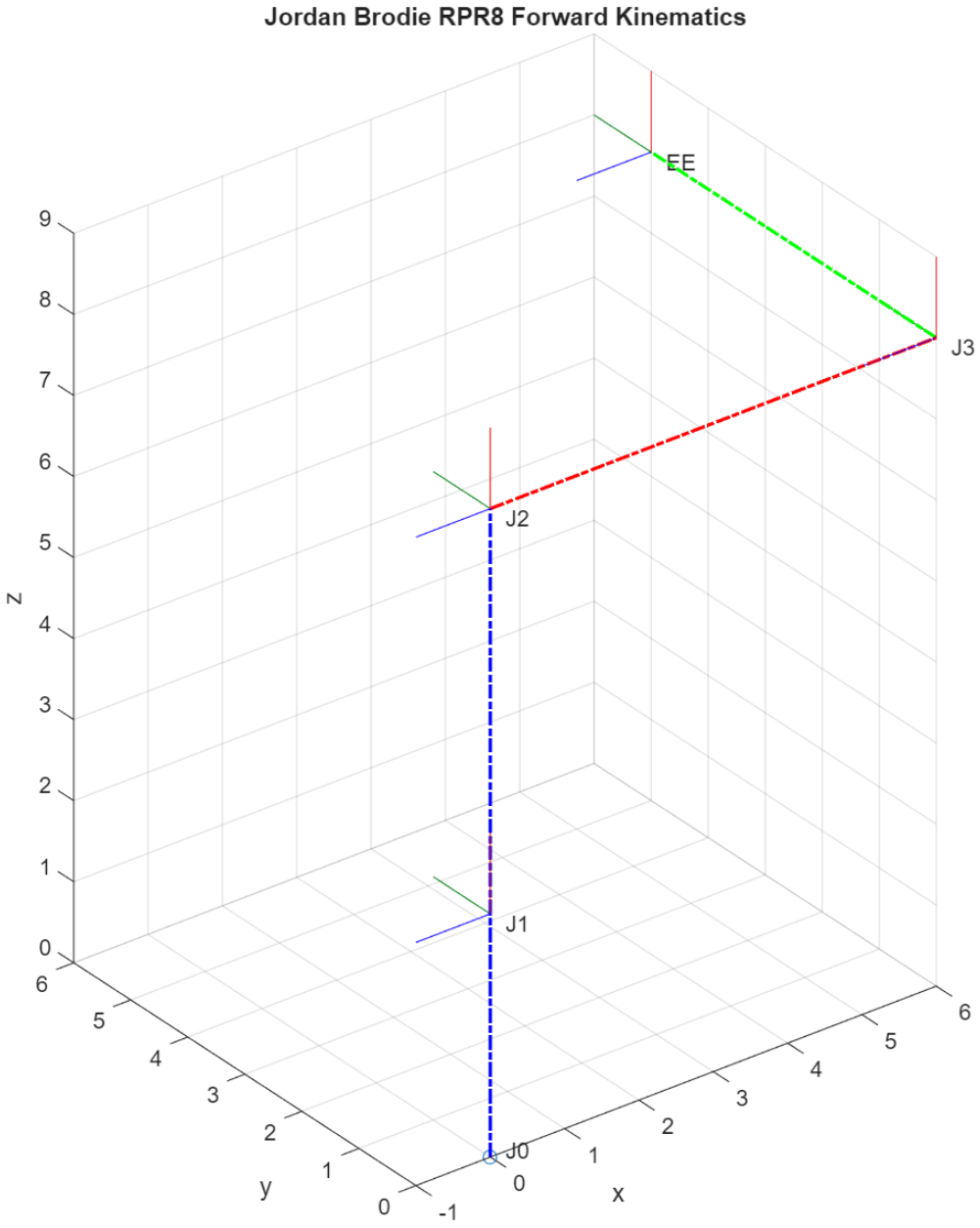


Figure 2: MATLAB plot of the RPR8 home configuration.

The generated plot agrees with the intended geometry of the RPR8 manipulator. J1 and J2 lie

vertically above the base. After the frame rotation, the prismatic link extends from J2 to J3, and the final link connects J3 to the end effector.

8 Conclusions

In this assignment I created the forward kinematics model for my RPR8 manipulator in MATLAB.

The hardest part for me was understanding the rotation from J0 to J1 and how to set up the transformation correctly in MATLAB. At first, it was confusing to understand which axis the J0 frame needed to rotate around and how that rotation would affect the following joints. Once I understood that the frame needed to rotate around the y-axis, the rest of the model became much clearer.

I also had to learn how the MATLAB transformation functions work and how to combine the translations and rotations in the correct order. This exercise was helpful because it showed me how the frame of reference changes from one joint to the next. Instead of thinking about every movement relative to the original base frame, I now better understand that each transformation can be defined relative to the previous joint's coordinate frame.

The final MATLAB model produced the expected home configuration, so I was able to confirm that the transformations and frame orientations were set up correctly.